R3st0

Première séance
Reprise du projet

Durée : 4h

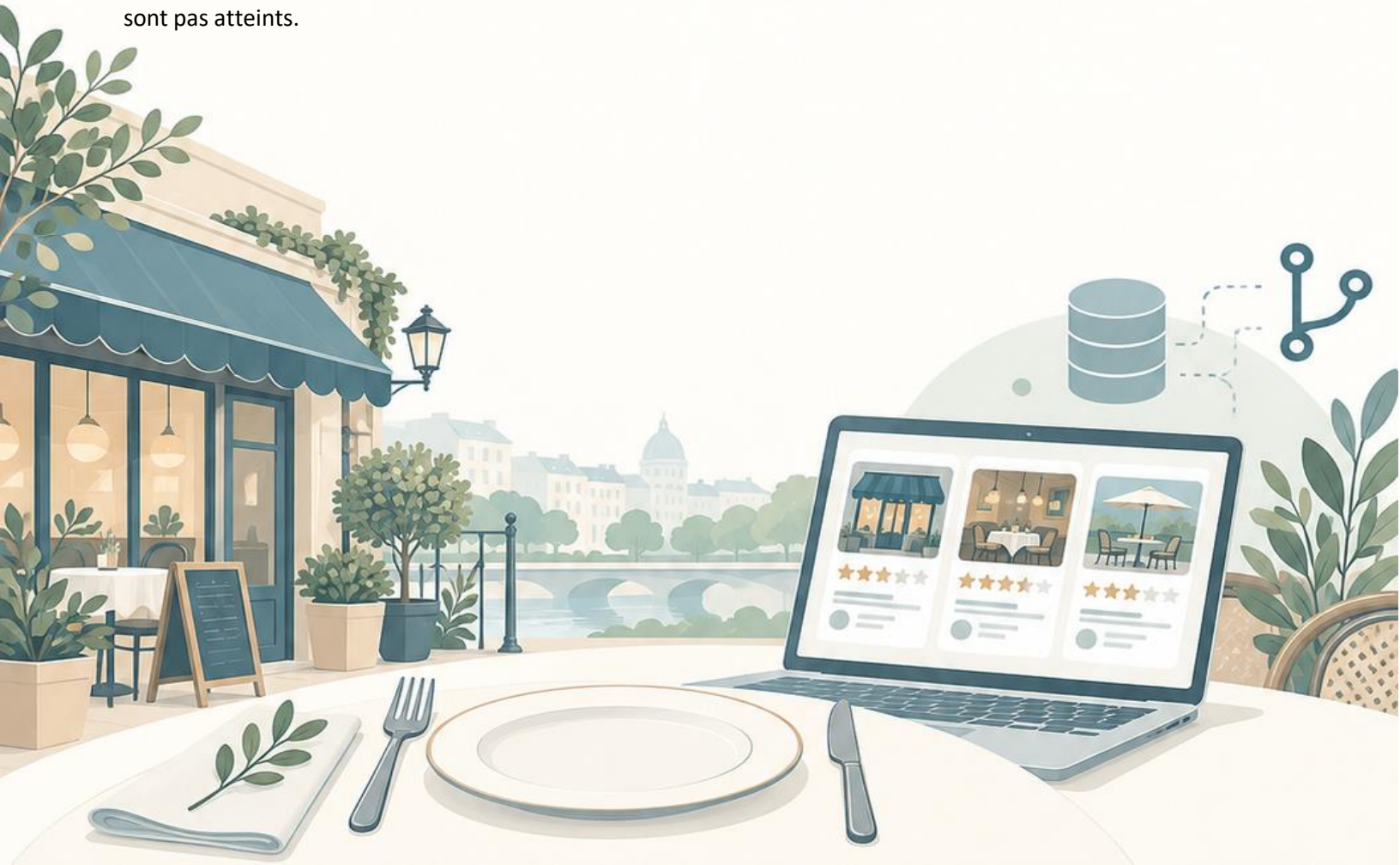
Contexte : Vous reprenez le développement de R3st0.fr, un site de critiques de restaurants. Le code PHP et les données existent déjà. Votre première mission consiste à faire fonctionner cette version, comprendre son organisation et préparer sa maintenance avec votre équipe.

La règle commune pour chaque contribution

Les contributions de code, de SQL et de documentation suivent le même cycle : branche depuis dev à jour, commits, mise à jour de dev dans la branche de travail, tests, validations, push et MR vers dev. Utilisez le TP Git en équipe, sections 12 à 15, pour retrouver les manipulations.

La description de MR précise ce qui change, pourquoi et comment tester. Ajoutez Refs #numéro pour relier l'Issue. Un autre membre lit les différences et teste la branche proposée. À trois, alternez les relecteurs. Si dev évolue pendant la revue, actualisez votre branche et retestez.

Après validation et intégration dans dev, fermez manuellement l'Issue. Les tâches sans modification de fichier, comme la création du tableau, se valident sur leur preuve. Conservez ouvertes les Issues dont les critères ne sont pas atteints.



Organisation de l’équipe et du dépôt

Ticket I1-01 - Organiser l’équipe et le dépôt

L’enseignant porte les besoins et les priorités du projet. Choisissez un animateur (Scrum Master) dans votre groupe pour faciliter les échanges. Chaque membre participe aux réalisations, aux tests et à la documentation. Le rôle technique GitLab ne remplace pas cette organisation.

Un seul dépôt partagé

1. Vérifiez vos accès Git et SSH déjà configurés. Utilisez le document de configuration Git et GitLab en cas de difficulté. NetBeans et VS Code sont autorisés.
2. Dans votre groupe GitLab privé, créez le projet P1_GX_SiteResto2026, où X identifie votre équipe. Un membre initialise le dépôt avec le contenu du dossier PhpProjectSiteRestoInitial. Conservez le README existant et ses crédits. Les autres membres clonent ensuite ce dépôt.
3. Donnez accès aux enseignants avec le rôle Maintainer. Un étudiant assure la maintenance du dépôt ; les autres disposent au minimum du rôle Developer. Après le premier import sur main, créez dev. Les modifications suivantes passent par des Merge Requests.

Avant le premier commit, excluez les réglages personnels de l’IDE, notamment nbproject/private/, avec un fichier .gitignore. Ne déposez ni clé privée, ni jeton GitLab, ni mot de passe personnel.

Le suivi dans GitLab

1. Créez le Milestone « Itération 1 - Reprise de l’existant », avec l’échéance du 21 septembre 2026. Créez les cinq Issues ci-dessous et rattachez-les à ce jalon.

Repère	Titre de l’Issue
I1-01	Organiser l’équipe et le dépôt
I1-02	Installer et lancer le site
I1-03	Diagnostiquer le fonctionnement et le MVC
I1-04	Comprendre les données
I1-05	Identifier l’équipe dans le pied de page

Les repères I1-01 à I1-05 sont des titres de suivi. Pour lier une branche ou une MR, utilisez le véritable numéro attribué par GitLab à l’Issue.

2. Pour chaque Issue, indiquez le travail attendu, ses critères de validation, un responsable et sa priorité. Utilisez le tableau À faire, En cours, À relire/tester, Terminé. Limitez le travail en cours et réorganisez-vous au point d’équipe.

Validation : chaque membre accède au dépôt ; main et dev existent ; les cinq Issues sont visibles. Conservez trois lignes sur votre objectif, la répartition initiale et les obstacles.

Installation de l'application

Ticket I1-02 - Installer et lancer le site

Ressource principale : R3st0_Archive_demarrage_Iteration1_2026.zip. Chaque étudiant travaille sur son clone du dépôt et sur sa propre base de développement.

La base à utiliser

Le site fourni attend resto2, présente dans le dossier sql de l'application. La base resto1 appartient à un exercice distinct sur l'ancien modèle de données. Sa migration ne conditionne pas le lancement du site.

1. Ouvrez le dossier contenant index.php dans votre IDE. Repérez controleur, modele, vue, sql et tests. Vérifiez que PHP et MariaDB sont disponibles, ainsi que le pilote PHP PDO MySQL.
2. À l'aide de votre outil SQL habituel et d'un compte disposant des droits nécessaires, importez sql/resto2.sql. Le script crée et sélectionne resto2. Vérifiez la présence des tables et des données. Ne réimportez pas le script dans une base déjà remplie sans examiner son état.
3. Examinez sql/resto_create_utilisateur.sql. Créez le compte applicatif local s'il n'existe pas et vérifiez ses droits sur resto2.
4. Vérifiez les paramètres de modele/dao/Bdd.class.php : serveur, base, identifiant et mot de passe de développement. Adaptez-les à votre environnement. Documentez les champs à renseigner sans publier vos secrets personnels.

Le lancement local

Utilisez le serveur PHP de votre environnement. Pour un lancement simple de développement, ouvrez un terminal dans le dossier contenant index.php, puis exécutez :

```
php -S localhost:8000
```

Gardez ce terminal ouvert. Dans le navigateur du même environnement que PHP, accédez à http://localhost:8000/. Si PHP tourne dans votre VM, ouvrez le navigateur dans cette VM. Arrêtez le serveur avec Ctrl+C.

Le serveur intégré convient ici aux essais locaux. PHP et MariaDB doivent fonctionner séparément. Un double clic sur index.php ou un serveur de fichiers statiques ne suffit pas à exécuter le code PHP.

La preuve attendue

Complétez le README : prérequis, scripts SQL dans l'ordre, paramètres à adapter et procédure de lancement. Chaque équipier vérifie la procédure sur son propre environnement.

Validation : accueil, liste et fiche d'un restaurant accessibles. Joignez une preuve représentative à l'Issue, avec l'URL, le résultat obtenu et les éventuelles difficultés.

Aide : manuel PHP, « Serveur web interne » — https://www.php.net/manual/fr/features.commandline.webserver.php

Diagnostic fonctionnel et architecture MVC

Ticket I1-03 - Diagnostiquer le fonctionnement et le MVC

Explorez la version fournie. Un défaut constaté devient une information pour la maintenance. Décrivez-le sans chercher à corriger toutes les anomalies pendant cette séance.

Les scénarios à vérifier

Scénario	Action à effectuer
Accueil	Ouvrir index.php et observer les restaurants affichés
Liste	Ouvrir index.php?action=liste
Fiche	Ouvrir une fiche depuis la liste et relever son URL
Recherche par nom	Chercher un nom présent puis un nom absent
Action inconnue	Ouvrir index.php?action=blob et observer la réponse

Dans docs/iteration1.md, consignez pour chaque scénario : action et données saisies, résultat attendu, résultat observé, conclusion. Un test sans résultat attendu ne permet pas de conclure. Conservez une capture lorsqu'elle aide à comprendre un résultat.

Pour toute anomalie, indiquez les étapes permettant de la reproduire. Vérifiez si un ticket équivalent existe avant d'en créer un. L'enseignant décidera avec vous de sa priorité pour la suite.

Une investigation sur la liste des restaurants

1. Partez de index.php?action=liste. Retrouvez comment le code choisit le contrôleur et relevez la valeur de $fichier juste avant son inclusion. Appuyez-vous sur le code ou sur une observation temporaire, à retirer ensuite.
2. Construisez un schéma simple du parcours : requête du navigateur, contrôleur frontal, contrôleur de l'action, DAO, base et vue. Utilisez les vrais noms de fichiers ou de méthodes. Distinguez les appels du retour des données.
3. Expliquez le rôle de modele/dao, modele/metier et vue. Retrouvez la méthode appelée pour obtenir la liste, la requête SQL correspondante et la variable reçue par la vue. Expliquez comment une classe peut être chargée sans require explicite dans le contrôleur en indiquant la classe qui s'en occupe.

Une vérification ciblée du DAO

1. Dans tests/dao/testRestoDAO.php, observez les parties getOneById et getAll. Comparez leurs résultats avec les données de la base resto2.

Validation : tableau de tests renseigné, parcours MVC expliqué et lien entre une donnée SQL et son affichage identifié. Chaque membre doit pouvoir expliquer le schéma.

Données et première contribution

I1-04 - Comprendre les données

1. Dans resto2, identifiez les clés primaires des tables utilisateur, resto, aimer et critiquer. Relevez les clés étrangères et expliquez comment elles relient utilisateurs, restaurants, avis et favoris.
2. Écrivez et testez la requête donnant le nom des restaurants et la note attribuée dans les critiques de l'utilisateur jj.soueix@gmail.com. Enregistrez-la dans sql/iteration1_requetes.sql et conservez son résultat dans votre bilan.
3. Ouvrez le script de structure dans le dossier ressources/resto1. Comparez la clé de la table utilisateur et les références dans aimer et critiquer avec resto2. Expliquez l'intérêt de idU et le rôle de l'unicité de mailU, notamment lorsqu'une adresse change.

Validation : clés et relations expliquées, requête exécutable sur resto2, résultat commenté et comparaison argumentée. Le script SQL « iteration1_requetes.sql » rejoint dev après relecture.

I1-05 - Identifier l'équipe dans le pied de page

Besoin : le pied de page doit afficher « Mise à jour par l'équipe X », où X est votre identifiant. La mention doit apparaître sur l'accueil, la liste et les fiches des restaurants, sans dégrader leur affichage.

1. Repérez le fichier partagé qui construit la fin des pages. Créez une branche depuis dev à jour, par exemple feature/pn-5-identification-equipe. Remplacez pn et 5 par vos initiales et le numéro réel de l'Issue.

2. Ajoutez la mention en respectant le HTML existant. Testez les trois pages concernées et vérifiez que leurs liens et contenus restent accessibles. Enregistrez des commits décrivant précisément la modification.

3. Actualisez dev, revenez sur votre branche et fusionnez dev dans cette branche. Résolvez les éventuels conflits et retestez avant de pousser. Ouvrez une Merge Request vers dev et demandez une relecture à un équipier.

Validation : mention correcte sur les trois pages, tests consignés et MR relue puis fusionnée. Le relecteur teste la branche proposée avant son intégration.

Approfondissement si le travail prioritaire est terminé

Sur une copie de resto1, réalisez le script de migration vers idU décrite dans l'ancien ticket de réingénierie. Conservez les données, l'unicité de mailU et les relations. Comparez les nombres de lignes et les résultats des jointures avant et après. Livrez un script séparé et vos preuves. Ne remplacez pas resto2 par cette base.

Intégration et remise de fin de séance

Les livrables dans le dépôt

Emplacement	Contenu attendu
README.md	Prérequis, import SQL, configuration et lancement reproductible
docs/iteration1.md	Membres, objectif et décisions ; tests fonctionnels ; schéma MVC ; analyse SQL ; obstacles et bilan
sql/iteration1_requetes.sql	Requête demandée, avec un commentaire sur son objectif
Issues et Merge Requests	Répartition, critères, preuves, tests et relectures
Tag et Release	Version de fin d'itération et notes sur les résultats et limites

Le bilan peut renvoyer aux Issues et MR pour éviter les duplications. Ajoutez une capture Git Graph commentée en fin d'itération. Préférez des explications courtes et vérifiables à une accumulation de captures.

La revue et la rétrospective

1. Vers 17h30, stabilisez dev et vérifiez ensemble le site. Proposez une MR de dev vers main. Faites-la relire et fusionner par un membre autorisé. Conservez dev pour la suite du projet, sans supprimer cette branche ni regrouper son historique par squash.

2. Créez le tag v0.1.0 sur main et une Release décrivant les résultats validés et les limites connues. Présentez à l'enseignant le site, votre schéma MVC et une contribution relue. Une Release identifie une version ; elle ne déploie pas le site sur un serveur.

3. Pendant les dix dernières minutes, notez dans le bilan : un point qui a bien fonctionné, une difficulté et une amélioration concrète pour le 28 septembre. Vérifiez les accès enseignants et rendez le lien du dépôt sur Moodle.

À 18h, poussez et documentez le travail fonctionnel réalisé, même incomplet. Indiquez ce qui reste à faire et pourquoi. Une difficulté expliquée aide l'équipe à préparer la suite.